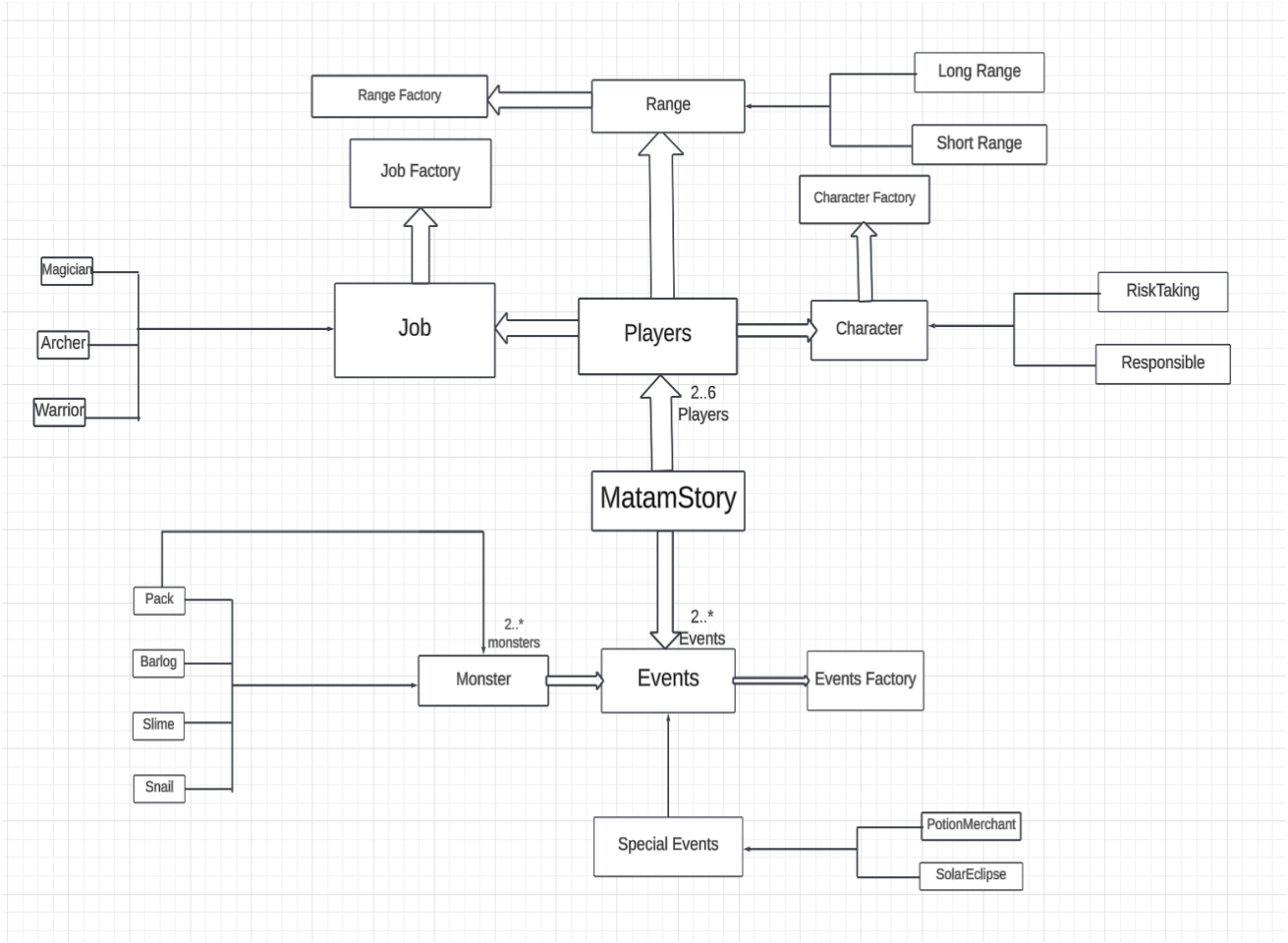


2.Strategy – התבנית באה לידי ביטוי בכך שהתנהגות השחקן (כמו עוצמת הקרב או תגובה למפגש עם סוחר) נקבעת לפי *האובייקטים* שהוא מחזיק מסוג Job, Range, ו־Character. כל אחד מהם מיישם ממשק שונה, וכך ניתן "להחליף" אסטרטגיה של שחקן בזמן ריצה מבלי לשנות את מבנה השחקן עצמו..

Composite – יישמנו תבנית זו במחלקה Pack, המחלקה יורשת שמייצגת קבוצה של מפלצות. מבנה זה מאפשר לנו להתייחס אל חבילה של מפלצות כמו אל מפלצת אחת, ובכך ניתן להתייחס אל Monster, מהמחלקה תומך ברקורסיה (חבילות בתוך חבילות) ומאפשר יישום אחיד של מפלצות בודדות וחבילות.

Factory – השתמשנו ב־Factory ליצירת אובייקטים מסוגים שונים של *אירועים*, *טווחים*, ו־*עבודות* של שחקנים. בצורה זו, הקוד מופרד מההחלטה איזה טיפוס אובייקט ליצור, והופך לגמיש *דמויות* ומודולרי יותר.

3.To implement the new job **Rogue**, we would follow these steps:

1. **Create a new class Rogue** that inherits from the existing Job class and overrides the relevant virtual functions.
2. In the Job class, we would add a new virtual function, for example applyMonsterEncounter(Player&, Monster&), with a default implementation that reflects standard combat behavior.
3. Inside the Rogue class, we would override this function such that if the monster's combat power is **at least twice** the player's combat power (monster.getCombatPower() >= player.getCombatPower() * 2), the player **escapes the encounter unharmed** (no damage is taken and no loot is gained).
4. If the condition is not met, the function would proceed with the default behavior (either by calling the base implementation or duplicating the logic).
5. In the JobFactory, we would add a condition that returns std::make_unique<Rogue>() when the input string is "Rogue", to support creation from input files.

4. Yes, our system can support this new feature.

We would proceed with the following additions and modifications:

Create a new class named DivineInspiration,

which inherits from SpecialEvent.

Implement the encounter behavior in the new class:

In DivineInspiration::applyEncounter(Player& player),

we will generate a random new job using the JobFactory (e.g., by selecting randomly from known job types).

Preserve accumulated bonuses:

If any job carries stateful bonuses (like the magician's force bonus from SolarEclipse),

we can store these bonuses in the Player object,

or design the Job interface to allow copying or transferring relevant state

via a function like transferBonusesFrom(const Job& previousJob).

Update JobFactory to support random job generation

(JobFactory::createRandomJob()), ensuring we don’t return the same job the player already has.

No changes are required in the, MatamStory, or Monster classes,

as we only a setJob func in Player Player::setJob().

and the behavior is encapsulated via the Strategy pattern.

So, the addition of the DivineInspiration event is fully supported by our code

with minimal changes.